

HW2 REINFORCE Report 2020203090 한용욱

본 과제에서는 정책기반 강화학습 알고리즘인 REINFORCE를 구현하고 `CartPole-v1` 과 `LunarLander-v3` 환경에서 할인율 γ 변화에 따른 성능 변화를 관찰하였다.

구현 방법

과제에서 제공된 스켈레톤 코드에서는 `Pi` 클래스의 `act` 메서드와 `train` 함수 내부의 리턴 계산 및 손실 정의 부분이 비워져 있었다. 이를 REINFORCE 알고리즘에 맞게 작성하였다.

정책 신경망 구조와 메모리

정책 신경망은 주어진 상태에서 각 행동에 대한 로짓을 출력한다. 에피소드 동안의 로그 확률과 보상은 다음 두 리스트에 저장한다.

- `log_probs`: 각 타임스텝에서 선택된 행동에 대한 $\log \pi_{\theta}(A_t | S_t)$
- `rewards`: 각 타임스텝에서 환경으로부터 받은 보상 R_t

이 두 리스트는 REINFORCE에서 한 에피소드에 대해 정책으로 생성한 데이터를 저장한다

`act` 함수 구현

에이전트가 환경으로부터 상태를 받았을 때, 정책으로부터 행동을 샘플링하고 해당 행동의 로그 확률을 저장하도록 `act` 함수를 구현하였다.

```
def act(self, state):
    # 모델 로짓 계산
    logit = self.forward(x)
    # 로짓을 이용해 Categorical 분포 생성
    pd = Categorical(logits=logit)
    # 정책에 따라 행동 샘플링
    action = pd.sample()
    # 이후 정책경사 계산에 사용할 log pi(a|s; theta) 저장
    selected_action_log_prob = pd.log_prob(action)
    self.log_probs.append(selected_action_log_prob)
    # Gym 환경에 넘겨줄 실제 행동 index 반환
    return action.item()
```

`forward`를 통해 얻은 `logit`은 각 행동에 대한 점수 벡터이다.

`Categorical(logits=logit)`을 사용하면 내부적으로 `softmax`가 적용된 것과 동일하게 확률분포로 해석할 수 있다.

`sample()`을 호출하면 $\pi_{\theta}(\cdot | s)$ 에서 행동 a 를 샘플링하게 되며, 그 때의 로그 확률 $\log \pi_{\theta}(a | s)$ 를 `self.log_probs`에 저장하여 이후 정책경사 계산에 사용한다.

train 함수 구현

한 에피소드가 끝난 뒤에는 `pi.rewards` 에 저장된 보상으로부터 각 시점의 리턴 G_t 를 계산하고, 이를 이용해 손실을 정의한다.

리턴은 $G_t = R_t + \gamma G_{t+1}$ 의 재귀식으로 계산한다. 이를 전개하면 다음과 같이 쓸 수 있다.

$$G_t = \sum_{k=t}^{T-1} \gamma^{k-t} R_k$$

코드에서는 다음과 같이 구현하였다.

```
def train(pi, optimizer):
    # 뒤에서부터 할인 누적합 계산
    for t in reversed(range(T)):
        G = gamma * G + pi.rewards[t]
        returns[t] = G
```

에피소드 단위의 목표 성능 함수는 $J(\theta) = E_{\pi_\theta}[G_0]$ 이며 정책경사 정리로 인해

$$\nabla_\theta J(\theta) = E_{\pi_\theta} \left[\sum_{t=0}^{T-1} G_t \nabla_\theta \log \pi_\theta(A_t | S_t) \right]$$

본 프로그램에서는 하나의 에피소드로 근사하기 때문에

$$\nabla_\theta \widehat{J(\theta)} = \sum_{t=0}^{T-1} G_t \nabla_\theta \log \pi_\theta(A_t | S_t)$$

딥러닝 프레임워크에서의 사용을 위해 위 경사가 나오도록 손실 함수를 정의하면 된다.

J 를 최대화해야하는것은 $-J$ 를 최소화하는것과 같으므로

$$L(\theta) = - \sum_{t=0}^{T-1} G_t \log \pi_\theta(A_t | S_t)$$

손실을 위와 같이 정의하면

$$\nabla_\theta L(\theta) = - \sum_{t=0}^{T-1} G_t \nabla_\theta \log \pi_\theta(A_t | S_t)$$

가 되어 $-\nabla_\theta J(\theta)$ 가 된다. 따라서 경사하강을 수행하면 실제로는 $J(\theta)$ 를 최대화하는 효과가 있다.

코드에서는 다음과 같이 구현하였다.

```
def train(pi, optimizer):
    # REINFORCE 목적함수에 대응하는 손실 정의
    loss = -(returns * log_probs)
    loss = torch.sum(loss)
```

벡터 형태의 `returns`와 `log_probs`를 원소별 곱한 뒤 전체 타임스텝에 대해 합을 취하면 위에서 정의한 손실식과 동일하다.

실험

실험 설정

환경은 `CartPole-v1` 과 `LunarLander-v3` 두 가지를 사용하였다.

정책 신경망 구조, 옵티마이저(`Adam`), 학습률 등 다른 설정은 고정하고 할인율 γ 만 변화시키며 실험하였다. 각 γ 값에 대해 학습된 정책으로 10 번의 테스트 에피소드를 실행하여 얻은 누적 보상의 평균을 기록하였다.

결과

아래 표는 할인율 γ 에 따른 두 환경에서의 평균 보상을 정리한 것이다.

`LunarLander-v3` 의 경우 보상은 소수점 둘째 자리에서 반올림하였다.

γ	<code>CartPole-v1</code>	<code>LunarLander-v3</code>
0.0	21.7	-222.9
0.0001	24.0	-189.0
0.1	29.0	-192.7
0.25	31.5	-230.4
0.5	29.5	-220.3
0.75	12.6	-720.2
0.9	48.6	-263.9
0.9999	107.2	-264.5
1.0	47.9	-156.2

해석

`CartPole-v1` 에서는 대체로 γ 가 너무 작을 때보다 어느 정도 크게 설정했을 때 평균 보상이 증가하는 경향을 보였다. 특히 아주 큰 할인율(예를 들어 $\gamma = 0.9999$)에서는 평균 보상이 크게 향상되었는데, 이는 막대를 오래 세워두는 장기적인 목표를 고려해야 좋은 정책이 되기 때문에, 미래 보상에 적절한 가중치를 주는 것이 유리하다는 점과 일치한다.

다만 학습 과정에서 보상이 크게 요동치기도 하고, 표에서 보이듯이 일부 γ 값(예를 들어 0.75)에서는 성능이 급격히 나빠지는 경우도 있었다. 이는 REINFORCE의 한계를 반영한다.

`LunarLander-v3` 의 경우 γ 가 커질때 보상이 증가하는 경향은 같으나 전반적으로 보상이 매우 불안정하며, `CartPole`에 비해 좋은 성능을 얻기 훨씬 어려웠다. 단순한 REINFORCE만으로는 안정적인 착륙 정책을 학습하기 어렵다는 것을 보여준다.